

Modules system (CommonJS vs ES6 modules)

Sure! Here's a detailed explanation of the **Modules System in Node.js**, focusing on **CommonJS** and **ES6 Modules**:

Node.js Module System

Node.js uses a modular architecture to separate code into reusable files. It supports two types of module systems:

1. CommonJS (CJS) Modules

- **Default in Node.js** (until v12, still widely used).
- **Uses** `require()` **and** `module.exports` for importing and exporting.

Syntax

```
// math.js
const add = (a, b) => a + b;
module.exports = { add };

// app.js
const math = require('./math');
console.log(math.add(2, 3)); // 5
```

Features

- **Synchronous loading** (best for server-side code).
 - Loads modules at runtime.
 - Easy to use with older Node.js projects and most npm packages.
-

2. ES6 Modules (ESM)

- **Modern JavaScript standard.**
- Uses `import` and `export` syntax.
- Supported in Node.js **from version 12+** (fully stable in 14+).

✓ Syntax

```
// math.mjs or package.json must set "type": "module"
export const add = (a, b) => a + b;

// app.mjs
import { add } from './math.mjs';
console.log(add(2, 3)); // 5
```

✓ Features

- **Asynchronous loading** (better suited for browsers).
- Static structure (known at compile time).
- Better for tools like bundlers, tree-shaking, and modern frameworks.

CommonJS vs ES6 Modules Comparison

Feature	CommonJS (CJS)	ES6 Modules (ESM)
Syntax	<code>require()</code> , <code>module.exports</code>	<code>import</code> , <code>export</code>
Loading	Synchronous	Asynchronous
Supported in	All Node.js versions	Node.js 12+ (stable in 14+)
File extension	<code>.js</code>	<code>.mjs</code> or <code>.js</code> with <code>"type": "module"</code>
Suitable for	Server-side, legacy code	Modern web and Node apps
Hoisting	No	Yes (ESM imports are hoisted)
Top-level <code>await</code>	✗ Not supported	✓ Supported

How to Use ES Modules in Node.js

To use ES modules in a `.js` file:

Option 1: Use `.mjs` extension

```
node app.mjs
```

Option 2: Set `"type": "module"` in `package.json`

```
{  
  "type": "module"  
}
```

Then use `.js` files with `import/export`.

Notes & Gotchas

- You **cannot mix** `require` and `import` in the same file (though interop is possible).
- Built-in modules like `fs`, `path`, etc., can be imported using either system.
- ESM modules **do not have** `__dirname` and `__filename` like CJS — use `import.meta.url` instead.

Let me know if you'd like examples of interoperability between CommonJS and ES Modules.